ASSIGNMENT 10

Name: Rohit Kailas Deore
Roll No: 11
Batch: T1
Class: TE-IT

[1]:
```
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

1. Load the Dataset

[2]:
```
# Use the built-in dataset; no Kaggle download is required
data = load_digits()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name="digit")

print("First five rows:")
display(X.head())
print("Dataset shape:", X.shape)
print("Number of features before PCA:", X.shape[1])
print("Target classes:", list(data.target_names))
```

```
First five rows:
```

	pixel_0_0	pixel_0_1	pixel_0_2	pixel_0_3	pixel_0_4	pixel_0_5	\
0	0.0	0.0	5.0	13.0	9.0	1.0	
1	0.0	0.0	0.0	12.0	13.0	5.0	
2	0.0	0.0	0.0	4.0	15.0	12.0	
3	0.0	0.0	7.0	15.0	13.0	1.0	
4	0.0	0.0	0.0	1.0	11.0	0.0	

	pixel_0_6	pixel_0_7	pixel_1_0	pixel_1_1	…	pixel_6_6	pixel_6_7	\
0	0.0	0.0	0.0	0.0	…	0.0	0.0	
1	0.0	0.0	0.0	0.0	…	0.0	0.0	
2	0.0	0.0	0.0	0.0	…	5.0	0.0	
3	0.0	0.0	0.0	8.0	…	9.0	0.0	
4	0.0	0.0	0.0	0.0	…	0.0	0.0	

```
   pixel_7_0  pixel_7_1  pixel_7_2  pixel_7_3  pixel_7_4  pixel_7_5  \
0        0.0        0.0        6.0       13.0       10.0        0.0
1        0.0        0.0        0.0       11.0       16.0       10.0
2        0.0        0.0        0.0        3.0       11.0       16.0
3        0.0        0.0        7.0       13.0       13.0        9.0
4        0.0        0.0        0.0        2.0       16.0        4.0

   pixel_7_6  pixel_7_7
0        0.0        0.0
1        0.0        0.0
2        9.0        0.0
3        0.0        0.0
4        0.0        0.0

[5 rows x 64 columns]
```

```
Dataset shape: (1797, 64)
Number of features before PCA: 64
Target classes: [np.int64(0), np.int64(1), np.int64(2), np.int64(3),
np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8), np.int64(9)]
```

2. Split the Dataset

[3]:
```python
# Split before scaling and PCA to prevent data leakage
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (1437, 64)
Testing data: (360, 64)
```

3. Scale the Features

[4]:
```python
# Fit the scaler only on training data
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Feature scaling completed.")
```

```
Feature scaling completed.
```

4. Apply PCA to Reduce Features to 2 Components

[5]:
```
# Fit PCA on training data only
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

print("Number of features after PCA:", X_train_pca.shape[1])
print("Explained variance ratio:", pca.explained_variance_ratio_.round(4))
print("Total variance retained:", round(pca.explained_variance_ratio_.sum(), 4))
```

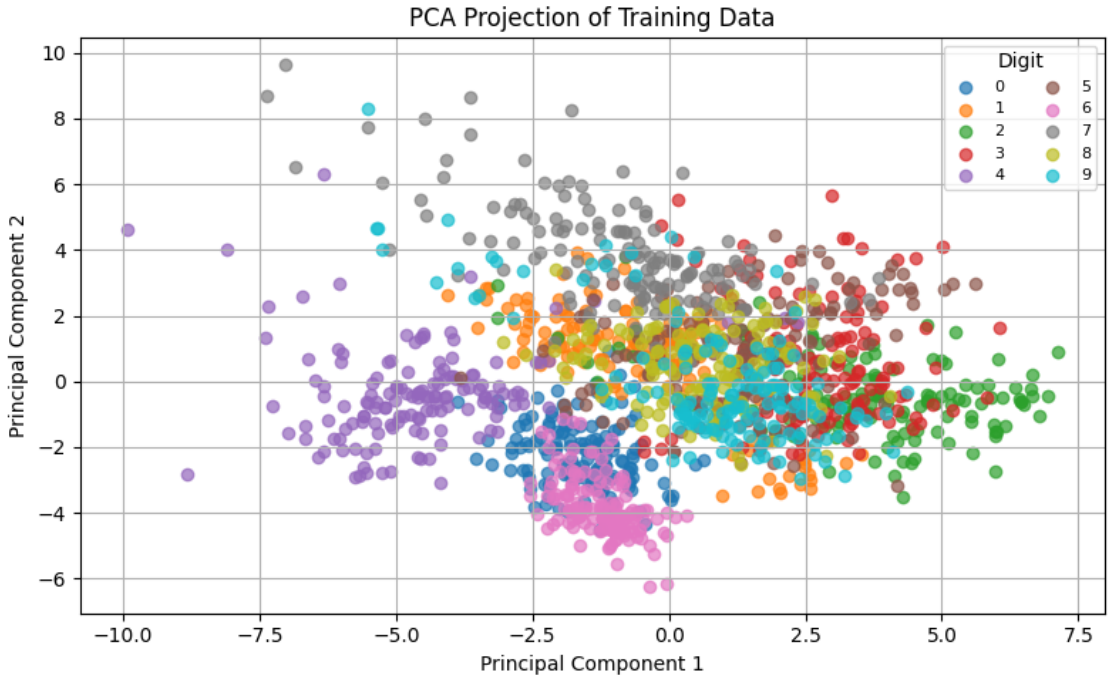
```
Number of features after PCA: 2
Explained variance ratio: [0.1204 0.0966]
Total variance retained: 0.217
```

5. Visualize the Two Principal Components

[6]:
```
plt.figure(figsize=(8, 5))

for label, class_name in enumerate(data.target_names):
    plt.scatter(
        X_train_pca[y_train.to_numpy() == label, 0],
        X_train_pca[y_train.to_numpy() == label, 1],
        label=class_name,
        alpha=0.7
    )

plt.title("PCA Projection of Training Data")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend(title="Digit", ncol=2, fontsize=8)
plt.grid(True)
plt.tight_layout()
plt.show()
```



6. Compare Logistic Regression Before and After PCA

[7]:
```
# Model using all original features
model_full = LogisticRegression(max_iter=2000)
model_full.fit(X_train_scaled, y_train)
pred_full = model_full.predict(X_test_scaled)
accuracy_full = accuracy_score(y_test, pred_full)

# Model using only two principal components
model_pca = LogisticRegression(max_iter=2000)
model_pca.fit(X_train_pca, y_train)
pred_pca = model_pca.predict(X_test_pca)
accuracy_pca = accuracy_score(y_test, pred_pca)

comparison = pd.DataFrame({
    "Model": ["Logistic Regression (All 64 Features)",
              "Logistic Regression (2 PCA Features)"],
    "Number of Features": [X_train_scaled.shape[1], X_train_pca.shape[1]],
    "Accuracy": [accuracy_full, accuracy_pca]
})

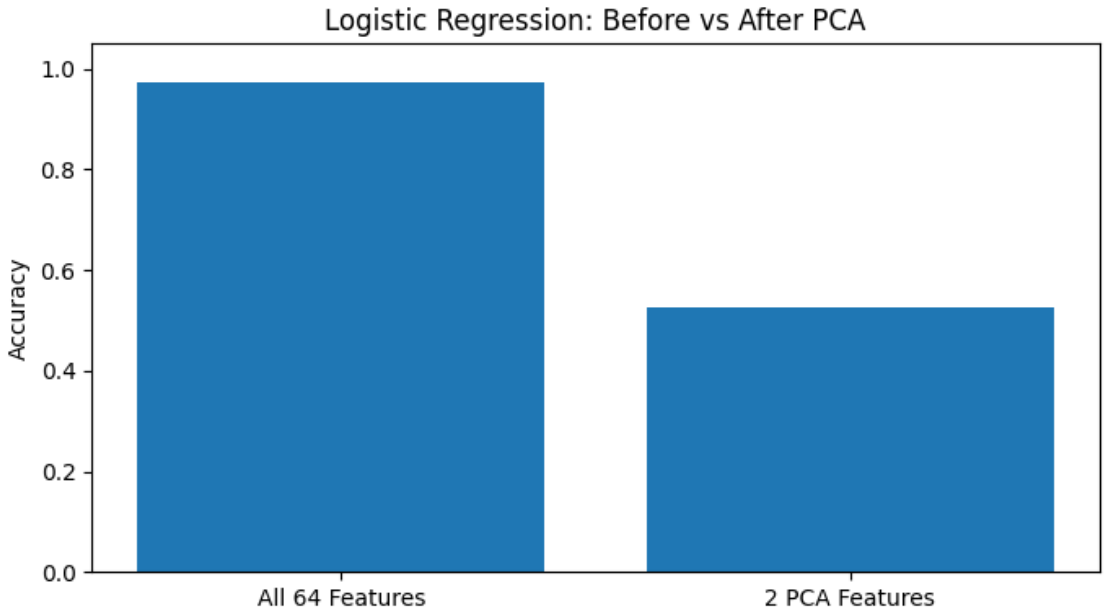
display(comparison.round(4))
print("Accuracy without PCA:", round(accuracy_full, 4))
print("Accuracy with PCA:", round(accuracy_pca, 4))
```

```
                                  Model  Number of Features  Accuracy
0  Logistic Regression (All 64 Features)                  64    0.9722
1   Logistic Regression (2 PCA Features)                   2    0.5250

Accuracy without PCA: 0.9722
Accuracy with PCA: 0.525
```

7. Visualize the Accuracy Comparison

[8]:
```
plt.figure(figsize=(7, 4))
plt.bar(
    ["All 64 Features", "2 PCA Features"],
    [accuracy_full, accuracy_pca]
)
plt.ylim(0, 1.05)
plt.ylabel("Accuracy")
plt.title("Logistic Regression: Before vs After PCA")
plt.tight_layout()
plt.show()
```



Conclusion

PCA was used to reduce the Digits dataset from 64 features to 2 principal components. Logistic Regression was evaluated using all original features and the two PCA features. The explained variance and accuracy comparison show the effect of dimensionality reduction. The scaler and PCA were fitted only on the training data to avoid data leakage.